the model.

But fear not. Practice has shown that it makes little difference whether we're looking at a database schema, a separate XML mapping file, or inline attributes in the model. The composite view is similar to the separations already happening in the model-view-controller pattern—just on a smaller scale.

Once the discomfort of treating the table schema as part of the model definition has dissipated, you'll start to realize the benefits of keeping DRY. When you need to add an attribute to the model, you simply have to create a new migration and reload the application.

Taking the "build" step out of schema evolution makes it just as agile as the rest of the code. It becomes much easier to start with a small schema and extend and change it as needed.

Instances of Active Record classes correspond to rows in a database table. These objects have attributes corresponding to the columns in the table. You probably noticed that our definition of class `Order` didn't mention any of the columns in the `orders` table. That's because Active Record determines them dynamically at runtime. Active Record reflects on the schema inside the database to configure the classes that wrap tables.

In the Depot application, our `orders` table is defined by the following migration:

rails7/depot_r/db/migrate/20221207000007_create_orders.rb

```
class CreateOrders < ActiveRecord::Migration[7.0]
  def change
    create_table :orders do |t|
      t.string :name
      t.text :address
      t.string :email
      t.integer :pay_type

      t.timestamps
    end
  end
end
```

Let's use the handy-dandy `bin/rails console` command to play with this model. First, we'll ask for a list of column names:

```
depot> bin/rails console
Loading development environment (Rails 7.0.4)
```